

CESIZen

PLAN DE DÉPLOIEMENT,
MAINTENANCE ET SÉCURISATION

Louis Beaujoin

CDA CESI · Bloc 3 · Juillet 2026

Déployer et sécuriser une application web en conditions professionnelles

48

TESTS CI/CD

v1.0.0

VERSION STABLE

12

VULNÉRABILITÉS P×I

Docker

PORT 8080 OPÉRATIONNEL

INFCDAAL3 — SOUTENANCE BLOC 3

1 Contexte & Enjeux	2 Architecture & Stack technique
3 Environnements DEV / TEST / PROD	4 Infrastructure Docker
5 Pipeline CI/CD GitHub Actions	6 Tests automatisés — 48 tests
7 Matrice de criticité (P × I)	8 Plan de sécurisation & chiffrement
9 Tests de sécurité automatisés	10 Gestion de crise & RGPD
11 Ticketing & SLA	12 Rollback & Résilience
13 Git Flow & Versioning	14 Démonstration live
15 Métriques & Conclusion	

Le projet CESIZen

Plateforme web de bien-être mental développée pour un **Ministère fictif** dans le cadre du CDA CESI.

Fonctionnalités

- Exercices de respiration guidés (3 prédéfinis + personnalisable)
- Pages d'information santé mentale
- Espace personnel avec historique de sessions
- Back-office d'administration
- API REST avec authentification par token

Objectif Bloc 3 — INFCDAAL3

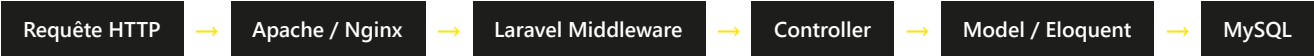
Déployer, maintenir et sécuriser une application web en conditions professionnelles :

- **Déploiement** : Docker · CI/CD · rollback · environnements
- **Sécurisation** : OWASP Top 10 · matrice P×I · gestion de crise
- **Maintenance** : ticketing · SLA · Dependabot · RGPD
- **Qualité** : 48 tests · Git Flow · SemVer · Conventional Commits

Résultat final — Juillet 2026

48/48 tests  · CI GitHub Actions  · Docker opérationnel  · github.com/louisbeaujoin/cesizen

Couche	Technologie	Version	Rôle
Framework	Laravel	13	MVC, routing, ORM Eloquent, auth, middleware
Langage	PHP	8.3	Backend — PHP-FPM en production
CSS / Build	Tailwind CSS + Vite	4 / 7	Utilitaires CSS · Hot reload DEV · Bundle PROD
Base de données	MySQL	8.0	Données persistantes · migrations Eloquent
Tests	Pest	4.x	BDD-style · 48 tests · SQLite in-memory (CI)
Serveur TEST	Apache (Docker)	2.4	Port 8080 · mod_rewrite · image build local
CI/CD	GitHub Actions	—	2 jobs séquentiels : tests → docker build



Critère	DEV (WAMP local)	TEST (Docker)	PROD (théorique)
Démarrage	composer dev	docker compose up --build	Pipeline CD automatisé
Port	8000	8080	443 (HTTPS — Let's Encrypt)
Base de données	MySQL WAMP	MySQL 8 container	MySQL managé (RDS)
APP_DEBUG	true	false	false
Assets Vite	Hot reload	Compilés dans l'image	Compilés + CDN
Ressources min.	4 Go RAM	4 Go RAM · 3 Go disque	2 vCPU · 4 Go · 50 Go SSD
Rollback	—	Volume drop + rebuild	git revert + mysqldump

L'environnement TEST Docker est celui réellement déployé et configuré dans ce projet (port 8080, données démo disponibles).

2 services — docker-compose.yml

Service app

- Image : build local (PHP 8.3 Apache + Node 20)
- Port : **8080** → **80**
- Entrypoint : entrypoint.sh

Service db

- Image : mysql:8.0
- Port : non exposé (réseau interne)
- Healthcheck : MySQL prêt avant l'app

Données démo — <http://localhost:8080>

admin@cesizen.fr · user@cesizen.fr / password

entrypoint.sh — démarrage automatique

```
# .env absent → copie le modèle (secrets hors image)
[ ! -f .env ] && cp .env.example .env

# Génère la clé applicative
[ -z "$APP_KEY" ] && php artisan key:generate --force

# Attend MySQL puis migre
until php artisan migrate --force; do
    sleep 3
done

# Seeder si demandé
if [ "${DB_SEED:-false}" = "true" ]; then
    php artisan db:seed --force
fi

exec apache2-foreground
```



Job	Déclencheur	Étapes principales	Résultat
tests	Push main / develop	Setup PHP 8.3 · Composer · SQLite · php artisan test	48/48 tests ✓
docker	needs: tests seulement	docker build -t cesizen-app .	Image buildée ✓

Dernière exécution : ✓ success — Juillet 2026 — ~1 min 44s

Si un test échoue → job docker annulé automatiquement. Zéro code cassé ne peut atteindre l'environnement TEST.

48

TESTS PASSÉS

122

ASSERTIONS

2.5s

DURÉE

0

ÉCHOUÉS

SQLite in-memory en CI

Pas de MySQL requis en CI — tests rapides et isolés.

Répartition des suites

Suite	Tests
Unit — Modèles (User, Breathing, Info)	9
Feature — Auth (connexion, rôles, profil)	13
Feature — Respiration (CRUD, sessions)	12
Feature — Information (pages, accès)	6
Feature — Sécurité (headers, throttle, XSS, CSP)	4
Exemples (sanity checks)	4
TOTAL	48

Méthode : Criticité = Probabilité (1–5) × Impact (1–5) · Score > 15 = Critique · 10–15 = Élevé · 5–9 = Moyen · <5 = Faible

#	Vulnérabilité	Réf.	P	I	Score	Niveau	Action implémentée
V01	Brute-force /connexion	A07	4	4	16	CRITIQUE	throttle:5,1 → HTTP 429 ✓
V02	Exposition secrets .env	A05	3	5	15	ÉLEVÉ	.gitignore + .dockerignore ✓
V03	Mode debug production	A05	3	4	12	ÉLEVÉ	APP_DEBUG=false ✓
V04	Injection XSS	A03	3	4	12	ÉLEVÉ	cast (int) + Blade + CSP ✓
V05	Mots de passe faibles	A02	3	4	12	ÉLEVÉ	bcrypt 12 rounds ✓
V06	Injection SQL	A03	2	5	10	ÉLEVÉ	Eloquent ORM ✓
V07	Accès routes admin	A01	2	5	10	ÉLEVÉ	Middleware auth + rôles ✓
V08	CSRF	A01	3	3	9	MOYEN	@csrf token ✓
V09–12	Session / Clickjack / MIME / CSP	A05/A07	2	2–4	4–8	FAIBLE–MOY.	5 headers HTTP · CSP ✓

✓ 12/12 vulnérabilités traitées — couverture 100% (grille : 75% minimum)

Prévention des risques

- **SecurityHeadersMiddleware** : 5 en-têtes HTTP (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy)
- **Throttle 5,1** : blocage brute-force → HTTP 429
- **Cast (int) XSS** : paramètres URL dans BreathingController
- **Eloquent ORM** : requêtes préparées PDO (SQL injection)
- **@csrf** : token sur tous les formulaires POST
- **Middleware auth + rôles** : admin vs user
- **APP_DEBUG=false** : aucune trace en production

Solutions de chiffrement

Donnée	Solution
Mots de passe	bcrypt 12 rounds (irréversible)
Sessions & cookies	AES-256-CBC via APP_KEY
Données en transit (PROD)	TLS 1.3 — HTTPS
Clé applicative	APP_KEY 32 octets — hors git
Tokens API	Hash SHA-256 en base

Contenus tiers bloqués par CSP

```
default-src 'self' ; object-src 'none' ; frame-ancestors 'none'
```

4 tests dans `tests/Feature/SecurityTest.php` prouvent que les mécanismes fonctionnent en CI :

```
// Test 1 – 5 en-têtes HTTP de sécurité sur toutes les pages (dont CSP)
test('security headers are present on home page', function () {
    $this->get('/')->assertHeader('X-Content-Type-Options', 'nosniff')
        ->assertHeader('Content-Security-Policy');
});

// Test 2 – CSP bloque les ressources externes
test('content security policy blocks external sources', function () {
    $csp = $this->get('/')->headers->get('Content-Security-Policy');
    expect($csp)->toContain("default-src 'self'")->toContain("object-src 'none'");
});

// Test 3 – HTTP 429 après 5 tentatives de connexion
test('throttle middleware blocks after 5 login attempts', function () {
    for ($i = 0; $i < 5; $i++) { $this->post('/connexion', [...]); }
    $this->post('/connexion', [...])->assertStatus(429);
});

// Test 4 – XSS neutralisé par cast (int) dans BreathingController
test('xss cast int protection on breathing page', function () {
    $this->get('/respiration?inspiration=5%3Balert(1)%2F%2F')
        ->assertStatus(200)->assertDontSee('alert(1)', false);
});
```

Vérification confirmée

Throttle testé sur Docker : HTTP 302×5 → HTTP 429  · En-têtes confirmés via Invoke-WebRequest 

Processus de réponse à incident

Délai	Action
H+0	Détection — logs, alertes Dependabot, ticket GitHub
H+15 min	Classification P1/P2/P3 · ticket ouvert
H+30 min	Escalade selon matrice (Direction + DPO si P1)
H+1h	Confinement — maintenance, révocation sessions
H+3h	Correction — branche hotfix/ → PR → merge → tag SemVer
<72h	Notification CNIL (Art. 33) + email utilisateurs (Art. 34)
J+7	Post-mortem documenté dans GitHub Issue

Conformité RGPD

Article	Mise en œuvre
Art. 5 — Minimisation	Nom, email, MDP hashé seulement
Art. 15 — Accès	Profil visible dans l'espace personnel
Art. 16 — Rectification	Modification depuis le profil
Art. 17 — Effacement	Suppression compte par admin
Art. 32 — Sécurité	bcrypt + HTTPS + headers + secrets hors git
Art. 33/34 — Violation	CNIL <72h · email utilisateurs <72h

GitHub Issues + Projects

Board Kanban — 4 colonnes

- Backlog → En cours → En revue → Terminé
- Templates pré-remplis (bug + feature)
- Labels : bug, amélioration, bloquant-critique, sécurité
- Fermeture automatique (Fixes #13)

Dependabot — veille automatique

- Composer + npm : vérification hebdomadaire
- GitHub Actions : vérification mensuelle
- PRs automatiques + validation CI avant merge

SLA par criticité

Niveau	Prise en compte	Correction
Bloquant critique	< 1h	< 3h ouvrées
Bloquant majeur	< 4h	< 24h ouvrées
Mineur	< 48h	Sprint suivant

Exemple bloquant critique

Erreur 500 sur /connexion → prise en compte immédiate, rollback si nécessaire, correctif <3h ouvrées.

Rollback code — Git

```
# Annuler le commit fautif (préserve l'historique)
git revert <hash>
git push origin main

# Revenir à un tag stable
git checkout v1.0.0
```

Rollback base de données

```
# Annuler la dernière migration
php artisan migrate:rollback

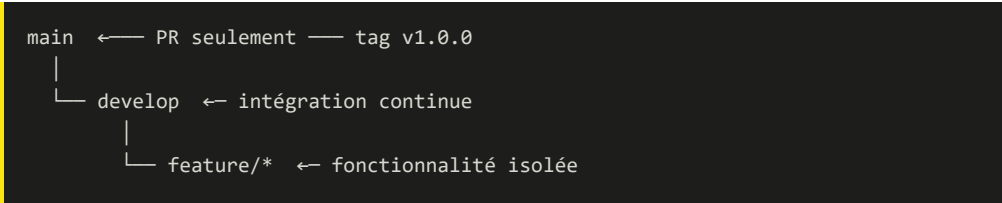
# Restaurer depuis backup
mysql -u root -p cesizen < backup.sql
```

Critères de déclenchement

Scénario	Délai
Erreur 500 en PROD	Immédiat
Régression majeure	< 30 min
Corruption données	Immédiat
Vulnérabilité critique	Immédiat + patch

Bonne pratique
mysqldump avant migration · Tags SemVer sur chaque release · git revert préféré au reset --hard

Architecture des branches



Flux de livraison



Semantic Versioning

Format	Signification
v1.0.0	Première version stable
v1.1.0	Nouvelle fonctionnalité (MINOR)
v1.0.1	Correctif de bug (PATCH)
v2.0.0	Changement cassant (MAJOR)

Conventional Commits

```
feat: nouvelle fonctionnalité
fix: correction de bug
docs: documentation
test: ajout de tests
chore: maintenance
```

A

VERSIONING + CI/CD

3 MIN

B

TICKETING GITHUB

2 MIN

C

APP DOCKER LIVE

1 MIN

6

MINUTES TOTALES

A — GitHub Actions

- Historique des commits
- Dernier run CI 
- 2 jobs : tests → docker
- Branches main / develop

B — Ticketing

- Board Kanban 4 colonnes
- Template bug rempli
- Labels criticité
- PRs Dependabot

C — App live

- `http://localhost:8080`
- Connexion admin (DevTools → en-têtes sécurité)
- Exercice de respiration
- Back-office administration

Si problème technique

- `docker compose restart app`
- `docker compose down -v && up --build`

48

TESTS PASSÉS

(0 ÉCHOUÉ)

122

ASSERTIONS

VÉRIFIÉES

12

VULNÉRABILITÉS

P×I ANALYSÉES

2

JOBS CI/CD

GITHUB ACTIONS

Critère grille	Objectif	Résultat
Déploiement	Environnement configuré + plan complet	✓ Docker 8080 · 3 env · CI/CD 2 jobs
Sécurité	OWASP + calculs criticité + plan	✓ 12 vulnérabilités P×I · 5 headers · CSP
Gestion de crise	Plan formalisé + RGPD	✓ 7 étapes · escalade P1/P2/P3 · CNIL 72h
Maintenance	Outil + SLA + veille	✓ GitHub Issues · SLA · Dependabot
Tests automatisés	Suite CI passante	✓ 48/48 · 122 assertions · 0 échec
Versioning	Outil + procédure	✓ Git Flow · SemVer v1.0.0 · Conventional Commits

CESI**Zen** — Bloc 3

Une application déployée, testée, sécurisée et maintenue
avec une chaîne d'outils professionnelle cohérente.

- 48 tests automatisés · CI GitHub Actions · 2 jobs séquentiels · 0 échec
- Docker opérationnel · <http://localhost:8080> · données démo incluses
- 12 vulnérabilités P×I · 5 headers HTTP · CSP · throttle · XSS · gestion de crise 7 étapes
- Ticketing GitHub · SLA formalisé · Dependabot hebdomadaire · rollback documenté
- Git Flow · SemVer v1.0.0 · Conventional Commits · PR develop → main
- RGPD Art. 5, 15, 16, 17, 32, 33, 34 · notification CNIL <72h

github.com/louisbeaujoin/cesizen · Louis Beaujoin · CDA CESI Bloc 3